

Best Practices for Data Synchronization Between Source Systems and Amazon OpenSearch

1 Executive Summary

The optimal approach for synchronizing data with Amazon OpenSearch depends primarily on latency requirements and source system type, but in nearly all cases, AWS-managed CDC pipelines should be the default starting point. For DynamoDB sources, the zero-ETL integration—which performs an initial snapshot via S3 and then streams ongoing changes through OpenSearch Ingestion (OSI) pipelines—delivers fully managed, near-real-time synchronization with no custom code [1, 2, 3]. For relational databases, AWS DMS with CDC replication provides equivalent low-friction synchronization with up to 49 parallel subtasks for initial loads [4, 5]. Event-driven architectures using Kinesis and Lambda remain appropriate only when complex custom transformations are required, as they introduce variable latency and higher operational overhead [6, 7].

The most impactful performance optimization is combining refresh interval suppression with bulk request sizing. Setting `refresh_interval` to `-1` during batch loads and targeting 5–15 MiB bulk requests eliminates segment creation overhead and maximizes throughput [8, 9]. For ongoing ingestion, shard sizing should follow workload type: 10–30 GiB per shard for search-sensitive workloads and 30–50 GiB for write-heavy analytics, with total shards not exceeding 500 per data node [10, 11].

Data integrity is best achieved through deterministic document IDs combined with at-least-once delivery, which produces effective exactly-once semantics without complex distributed transactions [12, 13]. Dead-letter queues writing to S3 are essential for all production pipelines, particularly DynamoDB integrations where schema mismatches can cause unexpected failures [13]. Cost optimization should leverage OR2 instances (up to 70% more indexing throughput than R7g) for active ingestion and UltraWarm (up to 90% storage cost reduction) for aging data [14, 15], with rightsizing targets of 60–80% CPU utilization to avoid over-provisioning.

2 Introduction

Keeping Amazon OpenSearch Service in sync with upstream source systems—relational databases, NoSQL stores, data lakes, and streaming platforms—is a foundational challenge in modern data architectures. Whether powering full-text search, real-time analytics dashboards, or observability platforms, the synchronization pipeline must balance latency, throughput, consistency, resilience, and cost. This report investigates the primary architecture patterns, AWS-native tooling, performance tuning strategies, consistency guarantees, error handling mechanisms, monitoring approaches, and cost optimization techniques for building robust data synchronization pipelines to Amazon OpenSearch. Practical recommendations are provided for both near-real-time and batch synchronization scenarios.

3 Architecture Patterns for OpenSearch Data Synchronization

Three primary architecture patterns dominate the landscape for synchronizing data into Amazon OpenSearch: Change Data Capture (CDC), batch ETL, and event-driven architectures. Each pattern presents distinct trade-offs across latency, operational complexity, flexibility, and data volume capacity.

Chart Explanation: This chart compares the three primary synchronization patterns across key dimensions. CDC/Zero-ETL excels in latency and data volume handling with minimal operational burden, batch ETL offers simplicity at the cost of freshness, and event-driven architectures provide maximum flexibility but require more operational investment.

Change Data Capture (CDC) is the recommended approach for most applications requiring near-real-time synchronization. DynamoDB provides two item-level CDC features—DynamoDB Streams and Amazon Kinesis Data Streams for DynamoDB—that replicate all item-level modifications to a data stream [16]. The DynamoDB zero-ETL integration with OpenSearch uses a snapshot-and-stream model: Point-in-Time Recovery (PITR) creates a backup uploaded to S3 for the initial load, then CDC synchronizes ongoing changes [2]. DocumentDB zero-ETL similarly streams full load and change data events via OpenSearch Ingestion pipelines using change streams [17]. AWS DMS supports OpenSearch as a target for both full-load and ongoing CDC replication from relational databases [4]. AWS recommends the zero-ETL integration as the primary approach for most applications due to its fully managed, no-code nature [1].

Batch ETL is best suited for large-volume periodic reindexing, data warehouse-style analytics, or scenarios where eventual consistency with hours-level staleness is acceptable. E-commerce workloads, for example, ingest product catalog updates via bulk APIs or real-time streaming depending on freshness requirements [18]. The trade-off is simpler implementation but data staleness and higher reindex costs at scale.

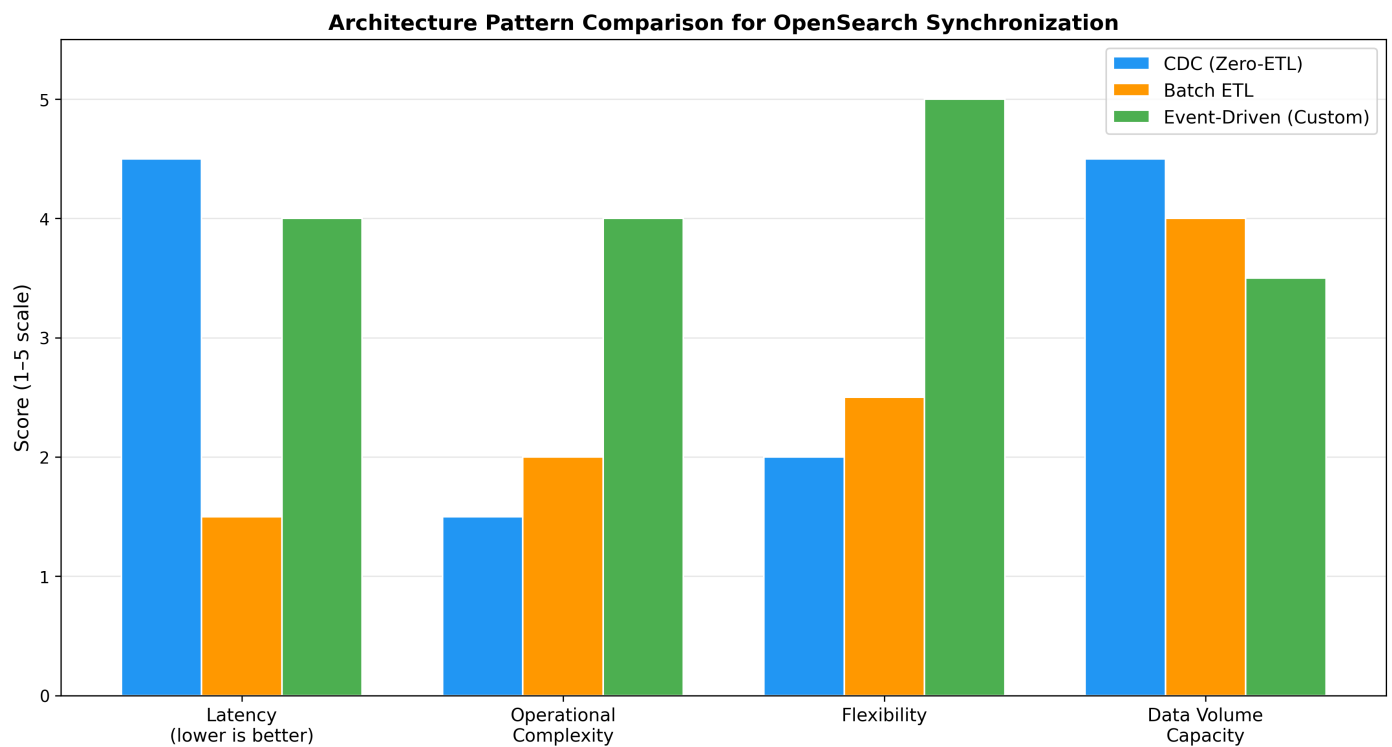


Figure 1: Architecture Pattern Comparison

Event-driven architectures offer maximum flexibility for complex transformations and multi-source fan-in. A common pattern routes DynamoDB changes through Kinesis Data Streams to Data Firehose, with Lambda performing transformations before indexing into OpenSearch in near real-time [6]. However, event-driven architectures introduce variable latency and eventual consistency trade-offs, making them unsuitable for workloads requiring consistent low-latency performance [7]. AWS DMS batch-apply mode can improve CDC replication performance for high-transaction workloads when custom event-driven pipelines are employed [19].

4 AWS Ingestion Tools and Integration Patterns

AWS provides a rich ecosystem of managed services for OpenSearch data ingestion, each suited to different source types and operational requirements.

OpenSearch Ingestion (OSI) pipelines are serverless, Data Prepper-based services configured via YAML with a source → processor(s) → sink architecture. Supported sources include HTTP/S push, Kafka (MSK and self-managed with mTLS, SASL/SCRAM, or IAM authentication), Amazon S3, OpenTelemetry Collector, Fluent Bit, and the DynamoDB plugin [20]. Pipelines with stateless processors scale up to 96 OCUs (or 384 OCUs with persistent buffering enabled), while stateful processors such as aggregate and service-map cap at 48 OCUs (192 with persistent buffering) [20]. The service handles scaling automatically, eliminating capacity planning overhead.

DynamoDB Zero-ETL Integration leverages the OSI DynamoDB plugin to perform an initial snapshot via DynamoDB export to S3, then replicate real-time changes using DynamoDB Streams with CDC for high-scale, low-latency streaming—requiring no custom Lambda code [2, 3]. This represents the lowest-friction path for DynamoDB-to-OpenSearch synchronization.

Kinesis Data Streams and Lambda remain the standard integration pattern for streaming sources that require custom transformation logic. Kinesis Data Streams and S3 use Lambda functions as event handlers to process and stream data to OpenSearch domains [21]. Throughput is bounded by Lambda concurrency limits and Kinesis shard count. Notably, OpenSearch 3.0 (released April 2025) introduced experimental pull-based ingestion directly from Kafka and Kinesis streams, potentially simplifying future architectures [22].

AWS DMS supports OpenSearch as a target endpoint for both full-load and ongoing CDC replication from relational databases. It supports multithreaded full loads with up to 49 parallel subtasks and 32 threads per table to increase transfer speed [5]. This makes DMS particularly effective for migrating relational data sources to OpenSearch with minimal custom code.

Tool	Best For	Scaling	Key Limitation
OpenSearch Ingestion (OSI)	Managed CDC, Kafka, S3 sources	Up to 384 OCUs [20]	Sink limited to OpenSearch
DynamoDB Zero-ETL	DynamoDB → OpenSearch sync	Automatic	DynamoDB-only source
Kinesis + Lambda	Custom transformations	Shard-based + concurrency	Lambda concurrency limits
AWS DMS	Relational DB migration/ CDC	Up to 49 subtasks [5]	Limited transformation

Table: AWS Ingestion Tools Comparison

5 Performance Optimization and Tuning Strategies

Optimizing OpenSearch ingestion performance requires careful tuning of bulk request sizing, refresh intervals, translog settings, and shard allocation strategies.

Bulk Request Sizing is the first lever for throughput optimization. AWS recommends starting with bulk requests of 5–15 MiB and incrementally increasing until indexing performance plateaus; some instance types limit bulk requests to 10 MiB [8]. Oversized bulk requests can cause memory pressure and rejected threads, while undersized requests underutilize network bandwidth.

Refresh Interval Tuning directly impacts indexing throughput. The AWS Well-Architected OpenSearch Service Lens recommends setting `refresh_interval` to 30 seconds or more to reduce resource contention from segment creation. During batch loads, disabling automatic refresh entirely (setting to -1) and re-enabling after completion yields significant throughput gains [9]. The default translog flush threshold of 512 MB can also be increased to reduce flush frequency, creating fewer large segments that merge less often and freeing threads for indexing [8].

Shard Sizing and Distribution require workload-aware configuration. The recommended shard sizes differ significantly based on access patterns.

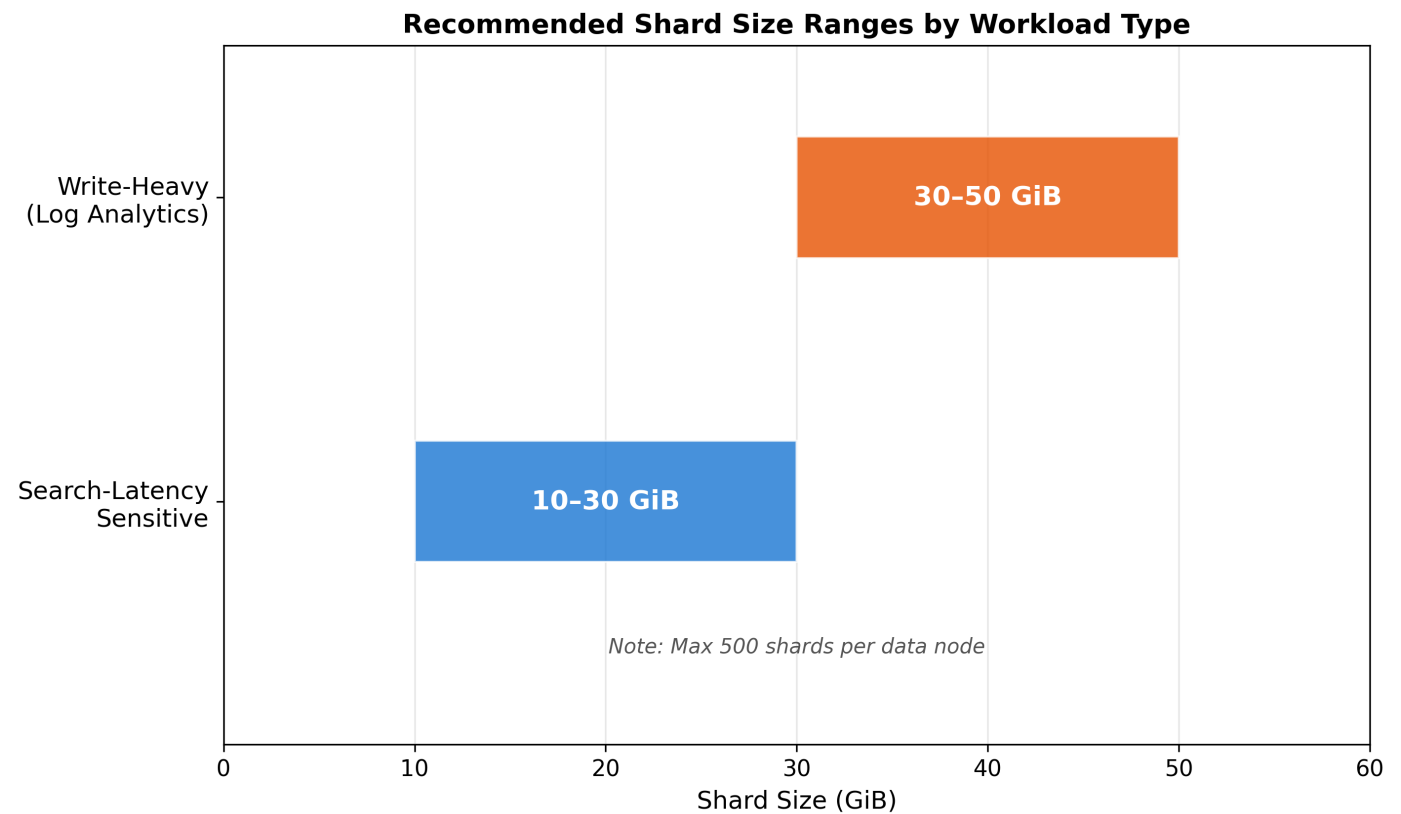


Figure 2: Shard Sizing Recommendations

Chart Explanation: This chart illustrates the recommended shard size ranges. Search-latency-sensitive workloads

should target 10–30 GiB per shard, while write-heavy workloads like log analytics can use larger 30–50 GiB shards. Clusters should not exceed 500 shards per data node [10].

The shard count formula is: $(\text{source data} \times 1.1) / \text{desired shard size} = \text{primary shards}$, and total shards (primary plus replica) should be configured as a multiple of the data node count for even distribution [11]. During heavy bulk indexing, setting `index.number_of_replicas` to 0 eliminates duplicate indexing work; replicas should be re-enabled after ingestion completes [8].

6 Consistency Guarantees and Data Integrity

OpenSearch synchronization pipelines operate under an eventual consistency model, but several techniques can strengthen data integrity guarantees.

Idempotent Writes for Effective Exactly-Once Semantics: OpenSearch Ingestion provides at-least-once delivery, meaning duplicate events may occur. However, OpenSearch’s Index Document API provides natural idempotency—using `PUT {index}/_doc/{id}` upserts the document, creating or updating it based on the ID [12]. For DynamoDB integrations, using the `primary_key` metadata as the `document_id` ensures updates are consistently replicated without conflicts [13]. This combination of at-least-once delivery with deterministic document IDs achieves effective exactly-once semantics in practice.

The Transactional Outbox Pattern addresses the challenge of maintaining ACID boundaries at the source while propagating changes to OpenSearch. The pattern ensures atomicity by writing both business data and an event record within a single ACID transaction, then using CDC (such as Debezium/Kafka Connector) to asynchronously propagate changes [23]. This architecture delivers eventual consistency with real-time synchronization between command and query models [24], preventing scenarios where data is written but events are not captured.

Replication Delay Monitoring serves as the primary reconciliation signal. AWS recommends setting alerts when replication delay exceeds approximately one minute, which provides adequate sensitivity without generating excessive noise [13].

7 Error Handling, Retry Strategies, and Resilience

Robust error handling is essential for production synchronization pipelines, as failures in distributed systems are inevitable rather than exceptional.

Dead-Letter Queues (DLQs) are the cornerstone of failure management. Amazon OpenSearch Ingestion pipelines support configuring a DLQ that writes failed events to an S3 bucket, enabling recovery without full index rebuilds [25]. The DLQ is configured in the sink section with `S3 bucket`, `key_path_prefix`, `region`, and `IAM role` parameters [26]. For DynamoDB integrations, AWS explicitly recommends always using a DLQ on OpenSearch sinks because DynamoDB is generally less structured than OpenSearch, making unexpected failures possible [13].

Retry Mechanisms are governed by redrive policies that define the maximum retry count, balancing adequate retries for transient failures against message accumulation in the queue [27]. The OpenSearch Bulk API processes actions independently—if one action fails, subsequent actions continue processing, and the response indicates per-action success or failure [28]. This enables selective retry of only failed documents rather than reprocessing entire batches, significantly improving recovery efficiency.

Reconciliation Strategies combine DLQ processing with replication delay alerts. When the delay threshold is breached, automated processes can trigger DLQ reprocessing or targeted re-synchronization of affected records. This layered approach—DLQ capture, selective retry, delay alerting, and automated recovery—ensures no data loss during synchronization failures [13].

8 Monitoring and Observability

Effective monitoring spans both the OpenSearch cluster and the ingestion pipeline, requiring attention to distinct metric categories.

Cluster Health Metrics published to CloudWatch at 60-second intervals include `CPUUtilization`, `JVMMemoryPressure`, `ThreadpoolWriteRejected`, `ThreadpoolSearchRejected`, and `IndexingRate`. `CPUUtilization` and `JVMMemoryPressure` should remain below 80% to avoid latency spikes and cluster instability [29]. Write thread pool rejections indicate the cluster cannot keep pace with ingestion volume and require either scaling or backpressure mechanisms.

Pipeline-Specific Metrics in the `AWS/OSIS` namespace include `recordsIn.count`, `recordsOut.count`, `timeElapsed.sum`, `PipelineLatency.avg`, and `PipelineLatency.max` for tracking synchronization lag and

throughput [30]. CloudWatch alarms should be configured to detect sustained latency increases or record count drops, which may indicate upstream source issues, transformation errors, or downstream cluster saturation.

Alerting Best Practices include setting replication delay alerts at approximately one minute [13], CPU/JVM pressure alerts at 80% [29], and monitoring the ratio of recordsIn to recordsOut for detecting silent data loss in the pipeline.

9 Cost Optimization and Resource Management

Cost optimization for OpenSearch synchronization workloads requires attention to instance pricing, storage tiering, instance selection, and rightsizing.

Reserved Instances and Savings Plans provide significant discounts over On-Demand pricing for predictable workloads. Reserved Instances are available with All Upfront, Partial Upfront, or No Upfront payment options for 1-year or 3-year terms. Database Savings Plans offer flexible commitment-based discounts that apply across instance families and regions [14].

Storage Tiering dramatically reduces costs for aging data. UltraWarm reduces cost per GB by up to 90% compared to hot storage for read-only data backed by S3 [15]. Data that no longer requires active indexing but still needs interactive query access should be migrated to UltraWarm, while cold storage provides even lower costs for rarely accessed data.

Instance Selection for Ingestion Workloads should consider the OR2 instance family, which delivers up to 70% more indexing throughput than R7g instances while eliminating replica storage costs via S3 [14]. For sustained ingestion workloads, optimal rightsizing targets are CPU utilization at 60–80%, JVM memory pressure at 65–85%, and storage utilization at 70–85%. Sustained metrics below 40% CPU, 50% JVM, and 50% storage indicate over-provisioning and wasted spend [14].

10 Practical Recommendations

For Near-Real-Time Synchronization, the recommended approach depends on the source system. DynamoDB sources should use the zero-ETL integration as the default [1], which provides a fully managed snapshot-and-stream pipeline via OSI with no custom code. For relational databases, AWS DMS with CDC replication offers the lowest operational overhead [4]. For sources requiring custom transformation, Kinesis Data Streams with Lambda provides the necessary flexibility [6]. In all cases, configure deterministic document IDs for idempotent writes [12, 13], enable DLQs [13], and set replication delay alerts at one minute [13].

For Batch Synchronization, disable automatic refresh (`refresh_interval: -1`) and set replicas to 0 before beginning the bulk load [9, 8]. Use bulk requests sized at 5–15 MiB [8] and increase the translog flush threshold beyond the 512 MB default [8]. After ingestion completes, re-enable replicas and restore the refresh interval. For relational sources, DMS multithreaded full loads with up to 49 subtasks provide efficient parallel transfer [5].

For Hybrid Architectures combining both patterns, maintain a CDC pipeline for ongoing near-real-time synchronization while scheduling periodic full reindexing jobs to correct any drift. Use separate index aliases to swap between the live index and the reindexed version atomically. Monitor pipeline metrics continuously [30] and leverage UltraWarm for historical data that no longer requires active indexing [15], ensuring cost efficiency across the entire data lifecycle.

11 References

1. <https://aws.amazon.com/solutions/guidance/real-time-text-search-using-amazon-opensearch-service/>
2. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/configure-client-ddb.html>
3. <https://aws.amazon.com/it/blogs/database/vector-search-for-amazon-dynamodb-with-zero-etl-for-amazon-opensearch-service/>
4. https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Target.Elasticsearch.html
5. https://docs.aws.amazon.com/en_us/dms/latest/userguide/CHAP_Target.Elasticsearch.html
6. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/4b457b09-f730-40ce-8e26-01d4427eb8c4?documentId=023a9ab2e-4bd6-ad7f-66b4e03cfda0>
7. <https://docs.aws.amazon.com/lambda/latest/dg/tradeoffs-event-driven.html>
8. <https://aws.amazon.com/premiumsupport/knowledge-center/opensearch-indexing-performance/>

9. <https://docs.aws.amazon.com/wellarchitected/latest/amazon-opensearch-service-lens/aosperf06-bp01.html>
10. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/bp-sharding.html>
11. <https://aws.amazon.com/blogs/big-data/amazon-cloudwatch-metrics-for-amazon-opensearch-service-storage-and-shard-skew-health>
12. <https://docs.opensearch.org/latest/api-reference/document-apis/index-document/>
13. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-integration-opensearch.html>
14. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/cost-optimization.html>
15. <https://aws.amazon.com/blogs/big-data/retain-more-for-less-with-ultrawarm-for-amazon-opensearch-service/>
16. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/4b457b09-f730-40ce-8e26-01d4427eb8c4?documentId=bb5f1707-297a-4e18-b8b8-353e1e9c81ef>
17. <https://docs.aws.amazon.com/documentdb/latest/devguide/openSearch-zero-etl.html>
18. <https://aws.amazon.com/blogs/big-data/amazon-opensearch-service-101-t-shirt-size-your-domain-for-e-commerce-search/>
19. <https://repost.aws/knowledge-center/dms-batch-apply-cdc-replication>
20. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/pipeline-config-reference.html>
21. https://docs.aws.amazon.com/en_us/opensearch-service/latest/developerguide/integrations.html
22. Source: wikipedia.org
23. <https://aws.amazon.com/blogs/mt/achieve-domain-consistency-in-event-driven-architectures/>
24. <https://aws.amazon.com/es/blogs/aws-spanish/cqrs-en-aws-sincronizando-los-servicios-de-command-y-query-con-el-estandar-transactional-outbox-la-tecnica-transaction-log-tailing-y-el-debezium-connector/>
25. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/osis-best-practices.html>
26. <https://opensearch.org/docs/2.10/data-prepper/pipelines/dlq/>
27. <https://aws.amazon.com/what-is/dead-letter-queue/>
28. <https://docs.opensearch.org/latest/api-reference/document-apis/bulk/>
29. <https://repost.aws/knowledge-center/opensearch-latency-spikes>
30. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/monitoring-pipeline-metrics.html>